

This actually makes sense: when the function got traced, the loop ran 10 times, so the `x += 1` operation was run 10 times, and since it was in graph mode, it recorded this operation 10 times in the graph. You can think of this for loop as a “static” loop that gets unrolled when the graph is created.

If you want the graph to contain a “dynamic” loop instead (i.e., one that runs when the graph is executed), you can create one manually using the `tf.nn.loop()` operation, but it is not very intuitive (see the “Using AutoGraph to Capture Control Flow” section of the Chapter 12 notebook for an example). Instead, it is much simpler to use TensorFlow’s *AutoGraph* feature, discussed in [Chapter 12](#). AutoGraph is actually activated by default (if you ever need to turn it off, you can pass `autograph=False` to `tf.function()`). So if it is on, why didn’t it capture the for loop in the `add_10()` function? It only captures for loops that iterate over tensors of `tf.data.Dataset` objects, so you should use `tf.range()`, not `range()`. This is to give you the choice:

- If you use `range()`, the for loop will be static, meaning it will only be executed when the function is traced. The loop will be “unrolled” into a set of operations for each iteration, as we saw.
- If you use `tf.range()`, the loop will be dynamic, meaning that it will be included in the graph itself (but it will not run during tracing).

Let’s look at the graph that gets generated if we just replace `range()` with `tf.range()` in the `add_10()` function:

```
>>> add_10.get_concrete_function(tf.constant(0)).graph.get_operations()
[<tf.Operation 'x' type=Placeholder>, [...],
 <tf.Operation 'while' type=StatelessWhile>, [...]]
```

As you can see, the graph now contains a `while` loop operation, as if we had called the `tf.nn.loop()` function.

Handling Variables and Other Resources in TF Functions

In TensorFlow, variables and other stateful objects, such as queues or datasets, are called *resources*. TF functions treat them with special care: any operation that reads or updates a resource is considered stateful, and TF functions ensure that stateful operations are executed in the order they appear (as opposed to stateless operations, which may be run in parallel, so their order of execution is not guaranteed). Moreover, when you pass a resource as an argument to a TF function, it gets passed by reference, so the function may modify it. For example:

```

counter = tf.Variable(0)

@tf.function
def increment(counter, c=1):
    return counter.assign_add(c)

increment(counter) # counter is now equal to 1
increment(counter) # counter is now equal to 2

```

If you peek at the function definition, the first argument is marked as a resource:

```

>>> function_def = increment.get_concrete_function(counter).function_def
>>> function_def.signature.input_arg[0]
name: "counter"
type: DT_RESOURCE

```

It is also possible to use a `tf.Variable` defined outside of the function, without explicitly passing it as an argument:

```

counter = tf.Variable(0)

@tf.function
def increment(c=1):
    return counter.assign_add(c)

```

The TF function will treat this as an implicit first argument, so it will actually end up with the same signature (except for the name of the argument). However, using global variables can quickly become messy, so you should generally wrap variables (and other resources) inside classes. The good news is `@tf.function` works fine with methods too:

```

class Counter:
    def __init__(self):
        self.counter = tf.Variable(0)

    @tf.function
    def increment(self, c=1):
        return self.counter.assign_add(c)

```



Do not use `=`, `+=`, `-=`, or any other Python assignment operator with TF variables. Instead, you must use the `assign()`, `assign_add()`, or `assign_sub()` methods. If you try to use a Python assignment operator, you will get an exception when you call the method.

A good example of this object-oriented approach is, of course, Keras. Let's see how to use TF functions with Keras.

Using TF Functions with Keras (or Not)

By default, any custom function, layer, or model you use with Keras will automatically be converted to a TF function; you do not need to do anything at all! However, in some cases you may want to deactivate this automatic conversion—for example, if your custom code cannot be turned into a TF function, or if you just want to debug your code (which is much easier in eager mode). To do this, you can simply pass `dynamic=True` when creating the model or any of its layers:

```
model = MyModel(dynamic=True)
```

If your custom model or layer will always be dynamic, you can instead call the base class's constructor with `dynamic=True`:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(dynamic=True, **kwargs)
    [...]
```

Alternatively, you can pass `run_eagerly=True` when calling the `compile()` method:

```
model.compile(loss=my_mse, optimizer="nadam", metrics=[my_mae],
              run_eagerly=True)
```

Now you know how TF functions handle polymorphism (with multiple concrete functions), how graphs are automatically generated using AutoGraph and tracing, what graphs look like, how to explore their symbolic operations and tensors, how to handle variables and resources, and how to use TF functions with Keras.

Symbols

@ operator (matrix multiplication), 135
 β (momentum), 380
 γ (gamma) value, 182
 ϵ (tolerance), 144, 183
 ϵ greedy policy, 706
 ϵ neighborhood, 279
 ϵ sensitive, 184
 χ^2 test, 203
 ℓ_0 norm, 45
 ℓ_1 norm, 45
 ℓ_2 norm, 45
 ℓ_k norm, 45

A

A/B experiments, 721
accelerated k-means, 268
accelerated linear algebra (XLA), 434
accuracy performance measure, 4, 107
ACF (autocorrelation function), 551
action advantage, reinforcement learning, 694
action potentials (APs), 301
actions, in reinforcement learning, 684, 693-694
activation functions, 306, 312-313
 for Conv2D layer, 490
 custom models, 415
 ELU, 363-366
 GELU, 365-367
 hyperbolic tangent (htan), 312, 565
 hyperparameters, 352
 initialization parameters, 360
 leakyReLU, 361-363
 Mish, 366
 PReLU, 362
 ReLU (see ReLU)
 RReLU, 362
 SELU, 364, 397
 sigmoid, 164, 312, 358, 651
 SiLU, 366
 softmax, 170, 315, 320, 600
 softplus, 314
 Swish, 366
active learning, 278
actor-critic, 717
actual class, 109
actual versus estimated probabilities, 118
AdaBoost, 223-226
AdaGrad, 382
Adam optimization, 384, 394
AdaMax, 385
AdamW, 386, 394
adaptive boosting (AdaBoost), 222-226
adaptive instance normalization (AdaIN), 671
adaptive learning rate algorithms, 382-387
adaptive moment estimation (Adam), 384
additive attention, 606
advantage actor-critic (A2C), 717
adversarial learning, 534, 636
affine transformations, 671
affinity function, 261
affinity propagation, 283
agents, reinforcement learning, 16, 684, 699, 705
agglomerative clustering, 282
Akaike information criterion (AIC), 289
AlexNet, 499
algorithms, preparing data for, 67-88

- alignment model, 606
- AllReduce algorithm, 760
- alpha dropout, 397
- AlphaGo, 17, 683, 716
- anchor priors, 528
- ANNs (see artificial neural networks)
- anomaly detection, 8, 13
 - clustering for, 262
 - GMM, 288-289
 - isolation forest, 293
- AP (average precision), 529
- APs (action potentials), 301
- area under the curve (AUC), 116
- argmax(), 171
- ARIMA model, 550-551
- ARMA model family, 549-551
- artificial neural networks (ANNs), 299-353
 - autoencoders (see autoencoders)
 - backpropagation, 309-312
 - biological neurons as background to, 301-302
 - evolution of, 300-301
 - hyperparameter fine-tuning, 344-353
 - implementing MLPs with Keras, 317-344
 - logical computations with neurons, 303
 - perceptrons (see multilayer perceptrons)
 - reinforcement learning policies, 691
- artificial neuron, 303
- association rule learning, 14
- assumptions, checking in model building, 37, 46
- asynchronous advantage actor-critic (A3C), 717
- asynchronous gang scheduling, 764
- asynchronous updates, with centralized parameters, 761
- à-trous convolutional layer, 533
- attention mechanisms, 578, 604-619, 625
 - (see also transformer models)
- attributes, 52-55
 - categorical, 71, 74
 - combinations of, 66-67
 - preprocessed, 55
 - target, 55
 - unsupervised learning, 11
- AUC (area under the curve), 116
- autocorrelated time series, 545
- autocorrelation function (ACF), 551
- autodiff (automatic differentiation), 785-791
 - for computing gradients, 426-430
 - finite difference approximation, 786
 - forward-mode, 787-790
 - manual differentiation, 785
 - reverse-mode, 790-791
- autoencoders, 635-659
 - convolutional, 648-649
 - denoising, 649-650
 - efficient data representations, 637-638
 - overcomplete, 649
 - PCA with undercomplete linear autoencoder, 639-640
 - sparse, 651-654
 - stacked, 640-648
 - training one at a time, 646-648
 - undercomplete, 638
 - variational, 654-658
- Autograph, 435
- AutoGraph, 806
- AutoML service, 349, 775
- autoregressive integrated moving average (ARIMA) model, 550-551
- autoregressive model, 549
- autoregressive moving average (ARMA) model family, 549-551
- auxiliary task, pretraining on, 378
- average absolute deviation, 45
- average pooling layer, 493
- average precision (AP), 529

B

- backbone, model, 504
- backpropagation, 309-312, 358, 428, 660
- backpropagation through time (BPTT), 542
- bagging (bootstrap aggregating), 215-219
- Bahdanau attention, 606
- balanced iterative reducing and clustering using hierarchies (BIRCH), 282
- bandwidth saturation, 763-765
- BaseEstimator, 80
- batch gradient descent, 142-144, 156
- batch learning, 18-19
- batch normalization (BN), 367-372, 565
- batch predictions, 731, 732, 739
- Bayesian Gaussian mixtures, 292
- Bayesian information criterion (BIC), 289
- beam search, 603-604
- Bellman optimality equation, 701
- bias, 155